Defense sheet — Assignment set 1

> Scope: this sheet covers the **coded** sub-questions only (2b, 2d, 2e, 3a, 3b, 3c). The hand-derivation parts (1a, 1b, 1c, 2a, 2c) are in the PDF and are not repeated here, but Fang can jump from code to derivation, so the cross-references below matter.

The task (in my own words)

Assignment 1 has three blocks. The graded coded pieces sit in blocks 2 and 3.

- **Block 2 — the COS method, sine variant.** The paper builds COS from a Fourier-**cosine** expansion; the assignment asks me to redo it with a Fourier-**sine** expansion.
  - **2b:** program the sine version of eq. (11) and reproduce the density-recovery errors of Table 1 of the COS paper for the standard normal density.
  - **2d:** implement the sine-COS European **put** pricing formula (derived by hand in 2c) under Black–Scholes parameters `S0=3, sigma=0.3, T=1, K=4, r=0.03`, and report the absolute error vs the analytic BS put as a function of `N`.
  - **2e:** extend the sine recovery one integration step further to recover the **CDF**, then verify it against the analytic log-normal CDF.
- **Block 3 — importance sampling for a tail quantile.** Portfolio of `N=1000` digital options; `L_T = sum_i 1{S_i(T) > K_i}` with a common Brownian factor `W` and idiosyncratic factors `B_i`. Estimate the 95% quantile of `L_T`.
  - **3a:** plain Monte Carlo.
  - **3b:** importance sampling, mean shift `W_T ~ N(1.5, T)`, reweight by `p/q`.
  - **3c:** importance sampling, mean + variance shift `W_T ~ N(1.5, 2T)`, reweight by `p/q`.

TODO (Bruno): In one sentence each, state the *point* of each sub-question — e.g. why 2b uses the standard normal density specifically, why 2e is "one extra integration step" beyond 2b/2c, and what the 3b-vs-3c comparison is meant to demonstrate about importance sampling.

My approach and why

What the code factually does (per block):

- **2b / 2e** build the sine series coefficients `F_k = (2/(b-a)) * Im{ phi(w_k) exp(-i w_k a) }` with `w_k = k*pi/(b-a)`, then sum `F_k` against either `sin(k pi (x-a)/(b-a))` (density, 2b) or the integrated sine `(1 - cos(...))` form (CDF, 2e).
- **2d** prices the put as `e^{-rT} * sum_k Im{ phi(w_k) exp(-i w_k a) } * V_k`, where `V_k` is the analytic sine coefficient of the put payoff built from the `phi_k`/`chi_k` primitives.
- **3a/3b/3c** simulate `S_i(T)` from the exact log-normal solution, count threshold crossings to form `L_T`, and take a (weighted, in 3b/3c) empirical 95% quantile.

TODO (Bruno): Justify, in your own words, the *modelling* choices that the code hard-codes: - 2b: why `[a,b] = [-10, 10]` is a safe truncation for the standard normal. - 2d: why you used the **cumulant-based** range `[a,b] = c1 ± L*sqrt(c2 + sqrt(c4))` with `L=10`, and why that is the "correct" way to set the range rather than a fixed window. - 2e: why you switched to a **fixed** window `[a,b] = [-2, 2]` for the CDF test instead of reusing the cumulant rule from 2d. - 3b/3c: why a mean shift of exactly `1.5`, and why doubling the variance is the natural "second" scheme to test.

Key code sections (file + what it does + why I wrote it that way)

2b — `2bCF (2).py` (sine density recovery, standard normal)

- `2bCF (2).py:4-5` set the truncation range `a=-10, b=10`.

- 2bCF (2).py:6-7 set the evaluation grid `X_test = arange(-5,6)` (i.e. integers -5..5) and `N_vals = [4,8,16,32,64]`.
- 2bCF (2).py:10-12 `normal_density(x)` returns the standard normal pdf `(1/sqrt(2pi)) exp(-x^2/2)`.
- 2bCF (2).py:14-16 `char_function(w)` returns `exp(-0.5 w^2)`, the characteristic function of `N(0,1)`.
- 2bCF (2).py:18-33 `sin_density(x_vals,a,b,N)`:
  - :22 `k = arange(1, N+1)` — sine series starts at k=1 (no k=0 term).
  - :23 `wk = k*pi/(b-a)`.
  - :26 `fk = (2/(b-a)) * Im{ char(wk) * exp(-i wk a) }` — the sine coefficients `F_k`.
  - :30-31 for each `x`, forms `sin(k pi (x-a)/(b-a))` and sums `fk * sin_terms`.
- 2bCF (2).py:39-43 loops over `N`, computes `max|f_approx - f_true|` and prints it.

TODO (Bruno): Explain *why* line 26 takes the **imaginary** part (tie it to the sine series and to Euler's formula in your 2a derivation), and why the coefficient prefactor is `2/(b-a)` and not `1/(b-a)`.

2d — `2dCF (2).py` (sine-COS European put pricing)

> **FACT / FORMATTING FLAG:** this file is stored as a **single physical line** (the newlines were stripped). So I cannot give honest per-line numbers here — I reference it by **function name**. Fang may notice this; be ready to say it is a save/encoding artefact, not the logic.

- Parameters block: `S0=3, sigma=0.3, T=1, K=4, r=0.03`; `x0 = log(S0/K)`; `mu = r - 0.5 sigma^2`.
- Cumulant range: `c1 = x0 + mu*T`, `c2 = T*sigma^2`, `c4 = 0`, `L = 10`, `a = c1 - L*sqrt(c2 + sqrt(c4))`, `b = c1 + L*sqrt(c2 + sqrt(c4))` (the comment cites "eq 49").
- `phi_BS(w, x, r, sigma, T)`: characteristic function of `y = ln(S_T/K)`, returns `exp(i w mu_y - 0.5 var_y w^2)` with `mu_y = x + (r - 0.5 sigma^2) T`, `var_y = sigma^2 T`.
- `phi_k(k,a,b,c,d)`: returns `(1/wk)*(cos(wk(c-a)) - cos(wk(d-a)))` — the `psi_k` primitive from 2c.
- `chi_k(k,a,b,c,d)`: returns `term(d)-term(c)` with `term(y) = e^y (sin(wk(y-a)) - wk cos(wk(y-a))) / (1 + wk^2)` — the `chi_k` primitive from 2c.
- `V_k_put(a,b,K,ks)`: returns `(2K/(b-a)) * (-chi_k(ks,a,b,a,0) + phi_k(ks,a,b,a,0))`, i.e. the payoff coefficient `V_k = (2K/(b-a))(psi_k(a,0) - chi_k(a,0))`.
- `sin_put_price(N,...)`: builds `cf_part = Im{ phi_BS(wk, x0, ...) exp(-i wk a) }`, multiplies by `Vk`, and returns `exp(-rT) * dot(cf_part, Vk)`.
- `bs_put(...)`: closed-form BS put for the reference price.
- The driver loops `N in [4,8,16,32,64,128,256,512,1024]` and prints `|SIN - BS|`.

> **NAMING FLAG (be ready for this):** in the report (2c) `psi_k` is the integral of `sin` and `chi_k` the integral of `e^y sin`. In this file the **function named `phi_k`** is the `sin` integral (the report's `psi_k`) and the function named `chi_k` is the `e^y sin` integral. So `phi_k` here = `psi_k` in the report. The numbers still match because `V_k_put` pairs them correctly, but the name `phi_k` collides with the characteristic function `phi`.

TODO (Bruno): Justify the *content* choices: why `c4 = 0` is acceptable here (what is the 4th cumulant of a Gaussian log-price?), why `x0 = log(S0/K)` is the right state variable, and why the discount factor is `e^{-rT}` rather than `e^{-r*Delta t}` with some other `Delta t`.

2e — `2eCF (2).py` (sine-COS CDF recovery)

- 2eCF (2).py:5-8 parameters `r=0.03, S0=3, sigma=0.3, T=1`.
- 2eCF (2).py:11-12 `mu = (r - 0.5 sigma^2) T`, `sigma_aux = sigma*sqrt(T)` — moments of `ln(S_T/S_0)`.
- 2eCF (2).py:14-15 fixed range `a=-2, b=2`.
- 2eCF (2).py:16-17 `X_test = linspace(-0.5, 0.5, 11)`, `N_vals = [4,8,16,32,64,128]`.
- 2eCF (2).py:19-23 `char_function(w)` = CF of `ln(S_T/S_0)`: `exp(i w mu - 0.5 sigma_aux^2 w^2)`.

- 2eCF (2).py:25-29 `F_exact(z) = norm.cdf((z - mu)/sigma_aux)` — analytic normal CDF.
- 2eCF (2).py:31-46 `sin_cdf(z_vals,N,a,b)`:
  - :36-37 `ks = arange(1,N+1)`, `wk = ks pi/(b-a)`.
  - :40 `im = Im{ char(wk) exp(-i wk a) }`.
  - :43 `cosine = cos( outer((z-a)/(b-a), ks) * pi )` — matrix of `cos(k pi (z-a)/(b-a))`.
  - :46 returns `(2/pi) * (im/ks) @ (1 - cosine).T`, i.e. `F(z) ~= (2/pi) sum_k (1/k) Im_k (1 - cos(k pi (z-a)/(b-a)))`.
- 2eCF (2).py:54-57 loop over `N`, print `max|F_sin - F_true|`.

TODO (Bruno): Explain *why* the CDF formula carries a `1/k` factor and a `2/pi` prefactor (where do they come from when you integrate the density term by term?), and why the `(1 - cos(...))` shape is the integrated form of `sin(...)`.

3a — `3aCF (2).py` (plain Monte Carlo quantile)

- 3aCF (2).py:5-12 parameters `N_stocks=1000, S_0=1, r=0.01, sigma1=0.8, sigma2=0.6, T=1, K=2, alpha = 1-0.95`.
- 3aCF (2).py:14 `drift = r - 0.5*(sigma1^2 + sigma2^2)`.
- 3aCF (2).py:16-31 `simulate_plainMC(paths)`:
  - :22 common factor `Wt ~ N(0, sqrt(T))` (shape `paths`).
  - :23 idiosyncratic `Bi ~ N(0, sqrt(T))` (shape `paths x N_stocks`).
  - :26 `S_T = S_0 exp(drift*T + sigma1*Wt[:,None] + sigma2*Bi)`.
  - :29 `L_T = sum(S_T > K, axis=1)` — count of in-the-money names per path.
- 3aCF (2).py:36 `np.random.seed(42)`.
- 3aCF (2).py:40-44 loops `paths` over `[1000..500000]`, `q = np.quantile(L_T, 1-alpha)` = 0.95 quantile.
- 3aCF (2).py:47-55 semilog convergence plot.

TODO (Bruno): Justify why `np.std`/`np.random.normal(0, sqrt(T))` (a `N(0,T)` draw) is the *exact* terminal law of `W_T` here, why no time-stepping is needed, and why the empirical quantile is taken at `1 - alpha = 0.95` given the assignment's definition `P(L_T >= q_alpha) <= alpha`.

3b — `3bCF (2).py` (importance sampling, mean shift)

- 3bCF (2).py:17-18 IS parameters `mu_shift = 1.5`, `mu_shiftsq = mu_shift^2 / (2T)`.
- 3bCF (2).py:20-34 `weighted_quantile(values, weights, q)`:
  - :25-27 sort `values`, reorder `weights` to match.
  - :30 `cdf = cumsum(w_sorted)/sum(w_sorted)`.
  - :33-34 `idx = searchsorted(cdf, q)`, return `v_sorted[idx]`.
- 3bCF (2).py:43 `W_q ~ N(mu_shift, sqrt(T))` — common factor drawn under `q`.
- 3bCF (2).py:44 `Bi ~ N(0, sqrt(T))` — idiosyncratic factors unchanged.
- 3bCF (2).py:47 `S_T = exp(drift*T + sigma1*W_q[:,None] + sigma2*Bi)` (note: no explicit `S_0`, since `S_0=1`).
- 3bCF (2).py:50 `L_T = sum(S_T > K, axis=1)`.
- 3bCF (2).py:53 `ratio = exp(-(mu_shift/T)*W_q + mu_shiftsq)` — likelihood ratio `p/q`.
- 3bCF (2).py:56 `q_95 = weighted_quantile(L_T, ratio, 0.95)`.

TODO (Bruno): Derive the `ratio` on line 53 from `p/q` of two Gaussians and confirm it equals the report's `exp(-1.5 w + 1.125)` at `T=1`. Also justify *why only `W` is reweighted and not `B_i`* (the report says the `B_i` ratio is 1 — state precisely why).

3c — `3cCF (2).py` (importance sampling, mean + variance shift)

- 3cCF (2).py:17-18 IS parameters `mu_shift = 1.5`, `var_q = 2*T`.
- 3cCF (2).py:20-34 `weighted_quantile` — identical structure to 3b.
- 3cCF (2).py:43 `W_q ~ N(mu_shift, sqrt(var_q))` = `N(1.5, 2T)`.
- 3cCF (2).py:44,47,50 same `Bi`, `S_T`, `L_T` construction as 3b.

- `3cCF (2).py:54 ratio = sqrt(2)*exp((-W_q^2 - 3 W_q + 2.25)/(4T))` — the `p/q` for the mean+var shift.
- `3cCF (2).py:56 q_95 = weighted_quantile(L_T, ratio, 0.95)`.

TODO (Bruno): Derive the line-54 ratio from `p ~ N(0,T)`, `q ~ N(1.5, 2T)` and confirm the `sqrt(2)` prefactor comes from the variance mismatch in the normalising constants. Then explain why this scheme is *worse* for the 95% quantile than 3b (the report's argument about mass landing too far in the tail) — restate it as your own.

Design decisions I must justify

- **Sine vs cosine expansion.** The code implements the sine series throughout (coefficients via `Im{...}`). **TODO (Bruno):** why does the assignment want sine, and what changes structurally vs the cosine COS (the `k=0` half-weight term, the `Im` vs `Re`)?
- **Truncation range, three different choices.** 2b uses `[-10,10]`, 2d uses the cumulant rule with `L=10`, 2e uses `[-2,2]`. **TODO (Bruno):** justify each range and explain whether `[-2,2]` in 2e is wide enough for `N(mu, sigma_aux^2)` with `mu = -0.015`, `sigma_aux = 0.3` (it is ~±6.7 sigma — confirm and say so). This is exactly the kind of "is your range safe?" question Fang asks.
- **`L = 10` cumulant multiplier. TODO (Bruno):** the paper uses `L` around 8–10; justify your `L=10` and what error you'd see if `L` were too small (link to your Assignment 2 short-maturity / small-c2 bug).
- **`np.quantile` interpolation vs the `weighted_quantile` searchsorted rule.** 3a uses NumPy's `np.quantile` (linear interpolation); 3b/3c use a custom step-function `searchsorted` quantile. **TODO (Bruno):** why two different quantile estimators, and does the mismatch bias the 3a-vs-3b/3c comparison?
- **`seed(42)`.** Used in 3a/3b/3c for reproducibility. **TODO (Bruno):** confirm this is only for reproducibility and does not, e.g., reuse the same `W` across schemes in a way that matters.

Results and what they mean

Reported in the PDF (factual, from the tables):

- **2b:** max density error falls `2.1e-1 (N=4) -> 1.0e-16 (N=64)` — matches the paper's Table 1 order of magnitude; spectral convergence.
- **2d:** `|SIN - BS|` falls to ~`3.3e-16` by `N=64` and stays there; BS reference put `= 0.9918191782`.
- **2e:** max CDF error falls `1.37e-1 (N=4) -> 3.67e-11 (N=32)` and then **plateaus** at `3.67e-11`.
- **3a:** 95% quantile estimate settles near `590` as `paths` grows to `5e5`.
- **3b:** estimate converges to ~`587` and visibly stabilises with far fewer paths than 3a.
- **3c:** estimate ends near `589` but the path is noisier; the report argues it is *less* efficient than 3b for this quantile.

TODO (Bruno): - 2d/2e: explain the **plateau** (why does the error stop improving — round-off vs truncation-range error?). For 2e specifically, why does it plateau at `3.67e-11` and not at machine `1e-16` like 2d? This is the single most likely "your convergence stalled — why?" question. - 3a/3b/3c: are 590 / 587 / 589 *consistent* (do their confidence intervals overlap)? What is the "true" quantile and how confident are you?

The hardest question Fang could ask here — and my answer

Likely hardest: *"Your 2e CDF error stalls at 3.67e-11 while your 2d price reaches 1e-16. Both use the sine-COS machinery. Why the difference, and is 3.67e-11 a bug or expected?"*

This bites because it sits exactly on the truncation-range / convergence theme Fang co-authored, and the two tables disagree by five orders of magnitude.

TODO (Bruno): Write the answer. Components you must cover: 1. The CDF formula integrates the density term by term, so its error inherits the **truncation-range** error `f(a), f(b) != 0` at the endpoints — i.e. the residual comes from `[a,b]=[-2,2]` not being infinitely wide, not from too few terms `N`. (Confirm by saying what happens if you widen `[a,b]`.) 2. Why 2d reaches machine precision instead: the put payoff coefficients `V_k` decay and the BS density is extremely well captured by the cumulant range, so series-truncation error drops below round-off. 3. State plainly whether `3.67e-11` is acceptable or whether widening `[-2,2]` would push it lower.

(Second candidate hardest: *"Derive line 53 / line 54 likelihood ratios from scratch and tell me why the variance shift in 3c hurts."* — see the 3b/3c TODOs.)

"Show me in your code where you do X" — anticipated spots

- **"…where you build the sine coefficients `F_k`."** -> `2bCF (2).py:26`; same construction at `2eCF (2).py:40` and inside `sin_put_price` (`cf_part`) in `2dCF (2).py`.
- **"…where you set the COS truncation range."** -> `2bCF (2).py:4-5` (fixed), cumulant block in `2dCF (2).py` (`a = c1 - L*sqrt(c2+sqrt(c4))`), `2eCF (2).py:14-15` (fixed `[-2,2]`).
- **"…where the characteristic function enters pricing."** -> `phi_BS` and `cf_part` in `2dCF (2).py`.
- **"…where you compute the payoff coefficients `V_k`."** -> `V_k_put` (with `phi_k`/`chi_k`) in `2dCF (2).py`.
- **"…where you integrate the sine to get the CDF."** -> `2eCF (2).py:43-46` (the `(1 - cosine)` term).
- **"…where you get the exact terminal stock price."** -> `3aCF (2).py:26`, `3bCF (2).py:47`, `3cCF (2).py:47`.
- **"…where you form `L_T`."** -> `3aCF (2).py:29`, `3bCF (2).py:50`, `3cCF (2).py:50`.
- **"…where you apply the likelihood ratio."** -> `3bCF (2).py:53`, `3cCF (2).py:54`.
- **"…where you take the weighted quantile."** -> `weighted_quantile` in `3bCF (2).py:20-34` / `3cCF (2).py:20-34`, called at `:56`.
- **"…where you take the plain quantile."** -> `3aCF (2).py:42` (`np.quantile(L_T, 1-alpha)`).